



Quantitative Project Risk Analysis with PRA

Paul Govan 
GE Aerospace

Abstract

The **PRA** package for R is the first package of its kind to unify a core set of widely adopted quantitative project risk analysis methods in a single, open-source, Comprehensive R Archive Network (CRAN)-available implementation: the Second Moment Method (SMM), Monte Carlo simulation (MCS), earned value management (EVM), contingency and sensitivity analysis, Bayesian risk inference and updating, sigmoidal learning curves, and design structure matrices (DSM). Beyond this foundational core, PRA makes two further contributions. As a secondary contribution, version 0.5.0 provides an open-source implementation of the causal- and probabilistic-network methods for project risk, supporting forward simulation of cost uncertainty and Bayesian conditioning along causal chains. As a third contribution, it exposes every analytical function as a tool over the Model Context Protocol (MCP), paired with a documented set of Agent Skills, so that large-language-model agents can run project risk analyses directly. The package adheres to rOpenSci Statistical Software Review standards and is supported by a continuous-integration test suite running across macOS, Windows, and Ubuntu.

Keywords: project risk analysis, Monte Carlo simulation, earned value management, Bayesian inference, design structure matrix, R.

1. Introduction

Quantitative risk analysis is a foundational discipline in project management, enabling organizations to assess schedule and cost uncertainty, identify high-impact risk drivers, and set evidence-based contingency reserves. This article presents **PRA**, an open-source R package that unifies the core quantitative methods of project risk analysis, schedule and cost uncertainty, earned value management, sensitivity and contingency analysis, learning curves, and dependency structure analysis, under a single, reproducible, CRAN-validated API.

Despite broad industry adoption of quantitative methods, open-source tooling for integrated project risk analysis in R remains fragmented. Proprietary platforms such as Oracle Pri-

mavera Risk Analysis and Palisade @RISK provide integrated workflows but impose licensing costs and limit reproducibility. Within R, no single package unifies Monte Carlo schedule risk, earned value management, Bayesian risk inference, sigmoidal learning curves, and dependency structure analysis under a common API. The **mc2d** package (Pouillot and Delignette-Muller 2010) provides general-purpose two-dimensional Monte Carlo simulation but has no project-specific constructs. The **sensitivity** package (Iooss, Da Veiga, Janon, and Pujol 2024) implements variance-based indices for general functions but does not integrate with schedule or cost models. EVM metrics are absent from CRAN entirely. Bayesian packages such as **rstan** (Stan Development Team 2024) and **brms** (Bürkner 2017) operate at a high level of generality and require substantial expertise to apply in project risk contexts.

The primary contribution addresses the gap above directly: To the author’s knowledge, **PRA** is the first CRAN package to integrate these project risk analysis methods in a single API. The comparison in Table~1 covers the closest general-purpose Monte Carlo and sensitivity packages and the dominant commercial platform; a broader survey of project-specific R packages is given below. Where the existing ecosystem is fragmented across general-purpose packages, none of them project-specific, **PRA** provides one reproducible, auditable API for the full workflow, integrating outputs across modules in a way the fragmented ecosystem does not. For example, MCS results feed directly into contingency and sensitivity analyses, and Bayesian risk updates can re-parameterize task distributions for subsequent simulations. The package targets project analysts who work in R and require outputs compatible with standard reporting pipelines. It adheres to rOpenSci Statistical Software Review (SRR) standards (rOpenSci Statistical Software Peer Review 2021) and uses minimal hard dependencies: **mc2d** (Pouillot and Delignette-Muller 2010) for triangular distribution families and **minpack.lm** (Elzhov, Mullen, Spiess, and Bolker 2016) for nonlinear least squares fitting.

As a secondary contribution, **PRA** provides an open-source R implementation—to the author’s knowledge the first—of the causal- and probabilistic-network methods for project risk introduced by Govan (2014) and Govan and Damjanovic (2016). That work grounds causal networks in the resource-based view, showing that organizational capabilities and shared resources—not schedule uncertainty alone—are the structural pathways through which risks propagate across project tasks. Govan and Damjanovic (2020) extend the approach with structural network measures that quantify the topology of project risk networks. The probabilistic network module (Section~7) operationalizes these ideas, propagating distributions through the causal chain from risks to total project cost.

As a third contribution, **PRA** is the first project risk analysis package, and to the author’s knowledge one of the first R packages in any domain, to expose its analytical methods as tools callable by LLM agents over the Model Context Protocol (MCP) and paired with a documented set of Agent Skills (Section~8). The optional MCP server activates through **Suggests**-only dependencies, so the analytical core carries no additional runtime cost.

The following table underscores the primary contribution: no existing R package or commercial platform covers the foundational project risk workflow that **PRA** unifies. It summarizes how **PRA** compares with the most closely related R packages and the dominant commercial platform.

A small number of CRAN packages do target project management specifically, but each addresses a single slice of the workflow rather than the integrated whole. The **Project-Management** package (Gonçalves-Dosantos, García-Jurado, and Costa 2022) is the closest

Table 1: Comparison of **PRA** with related R packages and commercial tools. “Partial” = only general-purpose support, requiring custom modeling and not project-specific; “—” = not supported. **PRA**’s MCS supports task correlation, but its current handling is approximate (Section 10).

Feature	PRA	mc2d	sensitivity	rstan/brms	@RISK
MCS schedule risk	Yes	Partial	—	—	Yes
EVM (11 metrics)	Yes	—	—	—	Yes
Bayesian updating	Yes	—	—	Yes	—
Sigmoidal learning curves	Yes	—	—	—	—
Design structure matrices	Yes	—	—	—	—
Probabilistic networks	Yes	—	—	Partial	—
MCP tool server	Yes	—	—	—	—
Open-source (CRAN)	Yes	Yes	Yes	Yes	—
SRR-compliant	Yes	—	—	—	—

in spirit: it implements deterministic and stochastic scheduling—PERT/CPM, critical-path identification, and Monte Carlo simulation over activity durations—and is complementary to the schedule-network extension planned for **PRA** (Section~10), but it does not provide earned value management, Bayesian risk updating, learning curves, dependency structure matrices, or probabilistic networks. The **plan** package (Kelley 2023) produces Gantt and burn-down charts for project visualization but performs no risk quantification. More generally, decision-analysis packages such as **decisionSupport** (Luedeling, Goehring, Schiffers, Whitney, and Fernandez 2023) apply Monte Carlo simulation to cost–benefit models under uncertainty, but are not organized around project-schedule or earned-value constructs. To the author’s knowledge, none of these packages—nor any single package on CRAN—spans the foundational project risk workflow that **PRA** unifies; this survey is not exhaustive, and a formal systematic review of the ecosystem remains future work.

The package is installed from CRAN in the standard way:

```
R> install.packages("PRA")
R> library("PRA")
```

Optional dependencies—**ellmer** (Wickham *et al.* 2025), **mcptools**, and **jsonlite**—are only required for the MCP server, `pra_mcp_server()`.

The remainder of this article describes each analytical module in turn, with working code examples drawn from the same R session. Section~2 covers schedule and cost uncertainty methods (SMM, MCS, contingency, sensitivity, and correlation matrix construction). Section~3 describes earned value management. Section~4 describes sigmoidal learning curves. Section~5 covers Bayesian risk inference and updating. Section~6 covers design structure matrices. Section~7 introduces the probabilistic network module. Section~8 describes the Model Context Protocol integration. Section~9 presents an integrated case study. Section~10 concludes.

2. Schedule and Cost Uncertainty

Two complementary methods are provided for quantifying schedule and cost uncertainty: the Second Moment Method for rapid analytical estimates, and Monte Carlo simulation for full distributional analysis. Contingency extraction and sensitivity analysis complete the workflow.

We illustrate the workflow with `building_project`, a bundled example dataset describing a mid-rise commercial building with six work packages, adapted from the construction-project examples in [Damnjanovic and Reinschmidt \(2020\)](#).

```
R> data("building_project", package = "PRA")
R> bp <- building_project
R> bp$task_names

[1] "Site Preparation & Earthworks" "Foundations"
[3] "Structural Frame"              "Building Envelope"
[5] "Mechanical/Electrical/Plumbing" "Interior Fit-out & Finishes"
```

2.1. Second Moment Method

The Second Moment Method (SMM) propagates task-level means and variances to project-level estimates using the Central Limit Theorem ([Benjamin and Cornell 2000](#)). For a project with n tasks:

$$E[X] = \sum_{i=1}^n E[X_i], \quad \text{Var}(X) = \sum_{i=1}^n \text{Var}(X_i) + 2 \sum_{i < j} \rho_{ij} \sigma_i \sigma_j$$

The six task durations are triangular, so we obtain their means and variances in closed form and propagate them with `smm()`:

```
R> tri_mean <- function(d) (d$a + d$b + d$c) / 3
R> tri_var <- function(d) (d$a^2 + d$b^2 + d$c^2 -
+ d$a * d$b - d$a * d$c - d$b * d$c) / 18
R> task_means <- vapply(bp$task_distributions, tri_mean, numeric(1))
R> task_vars <- vapply(bp$task_distributions, tri_var, numeric(1))
R>
R> result <- smm(task_means, task_vars) # independent tasks
R> cat("Total mean:", round(result$total_mean, 2), "weeks\n")
```

Total mean: 71.67 weeks

```
R> cat("Total SD: ", round(result$total_std, 2), "weeks\n")
```

Total SD: 4.53 weeks

Positive correlation between tasks inflates the total variance; `smm()` accepts a correlation matrix and adds the covariance terms analytically:

```
R> result_cor <- smm(task_means, task_vars, bp$cor_mat)
R> cat("Total SD (correlated):", round(result_cor$total_std, 2), "weeks\n")
```

```
Total SD (correlated): 6.59 weeks
```

SMM is useful when only first and second moments are available and a rapid estimate is needed. When task distributions are non-normal or accurate tail behavior is required, Monte Carlo simulation is preferred.

2.2. Monte Carlo Simulation

`mcs()` propagates uncertainty through a project network by drawing samples from user-specified task distributions, triangular, normal, or uniform, over a large number of iterations (Vose 2008). By default the tasks are sampled independently, where an optional correlation matrix is supported, though its current handling is approximate (Section~10).

```
R> set.seed(42)
R> results <- mcs(10000, bp$task_distributions)
R> cat("Simulated mean:", round(results$total_mean, 2), "weeks\n")
```

```
Simulated mean: 71.72 weeks
```

```
R> cat("Simulated SD: ", round(results$total_sd, 2), "weeks\n")
```

```
Simulated SD: 4.57 weeks
```

The `mcs()` function returns an S3 object of class "mcs" with a `print()` method that summarizes percentiles. The full simulated distribution is available in `results$total_distribution`. On a modern laptop, 10,000 simulations of a six-task network complete in under one second; increasing the count to 100,000 provides more precise tail estimates at roughly ten-fold additional runtime. The default of 10,000 balances accuracy and speed for most practical analyses.

Numerical Validation

For independent tasks the SMM estimates are the exact first two moments of the total: the total mean equals the sum of the task means by linearity of expectation, and the total variance equals the sum of the task variances by additivity of variance under independence. (The Central Limit Theorem governs only the asymptotic *shape* of the total, not the equality of its moments.) Monte Carlo simulation should therefore reproduce these moments, and the two agree to within Monte Carlo error, validating the sampling engine:

```
R> cat(sprintf("Mean: SMM %.2f vs MCS %.2f\n",
+             result$total_mean, results$total_mean))
```

```
Mean: SMM 71.67 vs MCS 71.72
```

```
R> cat(sprintf("SD: SMM %.2f vs MCS %.2f\n",
+             result$total_std, results$total_sd))
```

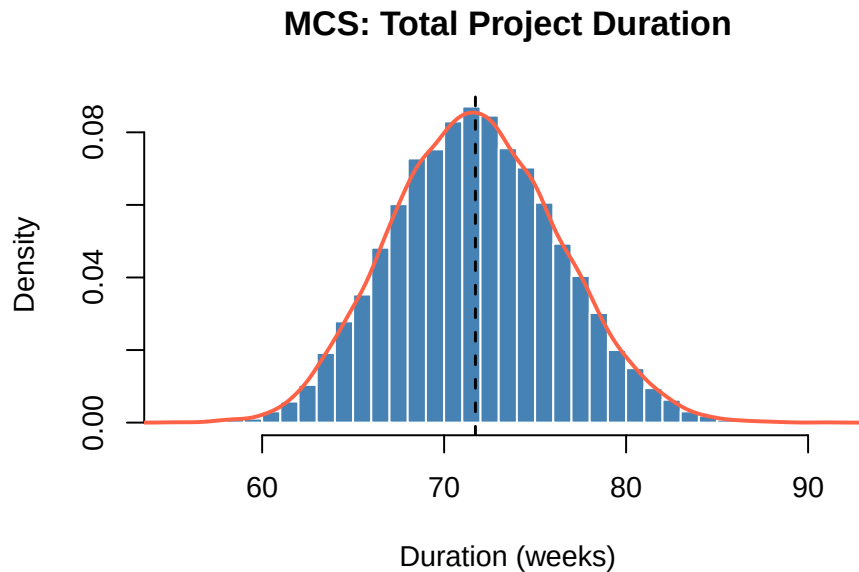


Figure 1: Distribution of simulated total project durations. The dashed vertical line marks the mean.

SD: SMM 4.53 vs MCS 4.57

```
R> hist(results$total_distribution,
+   breaks = 50, freq = FALSE,
+   main = "MCS: Total Project Duration",
+   xlab = "Duration (weeks)", col = "steelblue", border = "white"
+ )
R> lines(density(results$total_distribution), col = "tomato", lwd = 2)
R> abline(v = results$total_mean, lty = 2, lwd = 1.5)
```

2.3. Correlation Matrix Construction

Both `smm()` and `mcs()` accept a correlation matrix to model dependence between tasks. `cor_matrix()` constructs a valid positive-definite correlation matrix by empirically sampling from a set of task distributions, providing a convenient way to derive the correlation input from the same distributional assumptions used elsewhere in the workflow.

```
R> dists <- list(
+   normal = function(n) rnorm(n, mean = 10, sd = 2),
+   triang = function(n) mc2d::rpert(n, min = 5, mode = 10, max = 15),
+   uniform = function(n) runif(n, min = 8, max = 12)
+ )
R> C_emp <- cor_matrix(num_samples = 1000, num_vars = 3, dists = dists)
R> print(round(C_emp, 3))
```

```

      [,1] [,2] [,3]
[1,]  1.000 -0.028 -0.022
[2,] -0.028  1.000 -0.010
[3,] -0.022 -0.010  1.000

```

2.4. Contingency Analysis

`contingency()` extracts a reserve buffer as the difference between a high-confidence percentile and the base (median) estimate, a standard approach in project risk practice ([Project Management Institute 2021](#)).

```

R> reserve <- contingency(results, phigh = 0.80, pbase = 0.50)
R> cat("Schedule contingency (P80 - P50):", round(reserve, 2), "weeks\n")

```

```
Schedule contingency (P80 - P50): 3.98 weeks
```

2.5. Sensitivity Analysis

`sensitivity()` decomposes total project variance by task contribution, producing the inputs for a Tornado chart. Each task's index is its own variance plus its covariance with every other task, expressed as a proportion of total project variance.

```

R> sens <- sensitivity(bp$task_distributions)
R> names(sens) <- paste0("T", seq_along(sens))
R> barplot(sort(sens),
+   horiz = TRUE, col = "steelblue",
+   main = "Sensitivity: Task Contributions",
+   xlab = "Proportion of Total Variance"
+ )

```

Tasks with larger bars are the dominant drivers of total risk; mitigation resources should be directed there first.

3. Earned Value Management

The EVM module implements the complete ANSI/EIA-748 suite of eleven performance metrics ([Fleming and Koppelman 2010](#)). The three core quantities, Planned Value (PV), Earned Value (EV), and Actual Cost (AC), are computed first. All performance indices and forecasts follow from them.

```

R> bac <- 500000 # budget at completion ($)
R> schedule <- c(0.10, 0.25, 0.50, 0.75, 1.00) # cumulative value curve
R> time_period <- 3

```

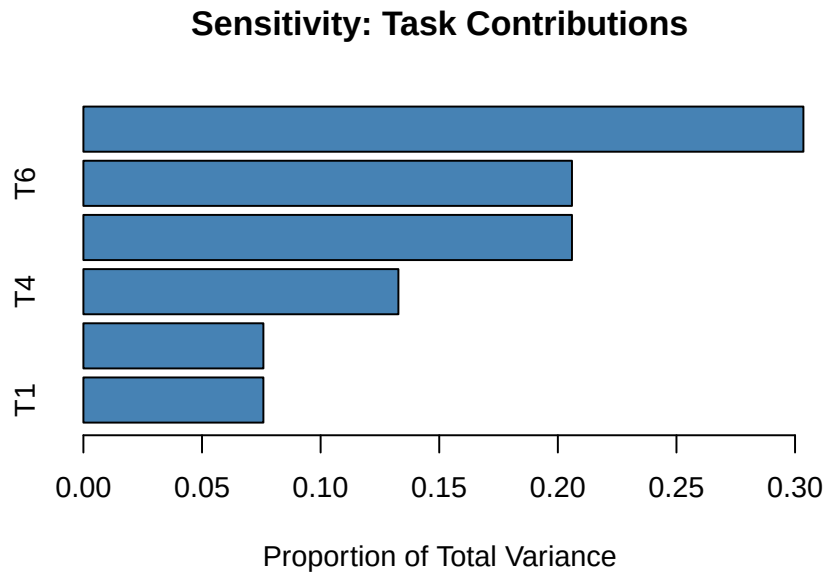


Figure 2: Tornado chart: each bar shows a task's share of total project duration variance.

```
R> pv_val <- pv(bac, schedule, time_period)
R> ev_val <- ev(bac, actual_per_complete = 0.40)
R> ac_val <- ac(c(45000, 110000, 135000), time_period, cumulative = FALSE)
R>
R> cat("PV: $", format(pv_val, big.mark = ","), "\n")
```

PV: \$ 250,000

```
R> cat("EV: $", format(ev_val, big.mark = ","), "\n")
```

EV: \$ 2e+05

```
R> cat("AC: $", format(ac_val, big.mark = ","), "\n")
```

AC: \$ 290,000

```
R> spi_val <- spi(ev_val, pv_val)
R> cpi_val <- cpi(ev_val, ac_val)
R> sv_val <- sv(ev_val, pv_val)
R> cv_val <- cv(ev_val, ac_val)
R>
R> cat("SPI:", round(spi_val, 3), " CPI:", round(cpi_val, 3), "\n")
```

SPI: 0.8 CPI: 0.69


```
R> cat("SV: $", format(round(sv_val), big.mark = ","),
+      " CV: $", format(round(cv_val), big.mark = ","), "\n")
```

```
SV: $ -50,000 CV: $ -90,000
```

Both $SPI < 1$ and $CPI < 1$ indicate the project is behind schedule and over budget. The `eac()` function provides three completion forecasts under different efficiency assumptions (Fleming and Koppelman 2010):

$$EAC_{\text{typical}} = \frac{BAC}{CPI}, \quad EAC_{\text{atypical}} = AC + (BAC - EV), \quad EAC_{\text{combined}} = AC + \frac{BAC - EV}{CPI \times SPI}$$

The typical method assumes remaining work proceeds at the cumulative cost efficiency to date; the atypical method assumes the overrun to date was a one-off and future work runs on plan; the combined method deflates the remaining budget by both cost and schedule performance.

```
R> cat("EAC (typical): $", format(round(eac(bac, method = "typical",
+      cpi = cpi_val))), big.mark = ","), "\n")
```

```
EAC (typical): $ 725,000
```

```
R> cat("EAC (atypical): $", format(round(eac(bac, method = "atypical",
+      ac = ac_val, ev = ev_val))), big.mark = ","), "\n")
```

```
EAC (atypical): $ 590,000
```

```
R> cat("EAC (combined): $", format(round(eac(bac, method = "combined",
+      cpi = cpi_val, ac = ac_val, ev = ev_val, spi = spi_val))),
+      big.mark = ","), "\n")
```

```
EAC (combined): $ 833,750
```

The remaining EVM metrics are:

```
R> eac_typ <- eac(bac, method = "typical", cpi = cpi_val)
R> cat("ETC: $", format(round(etc(bac, ev_val, cpi = cpi_val))), big.mark = ","), "\n")
```

```
ETC: $ 435,000
```

```
R> cat("TCPI:", round(tcpi(bac, ev_val, ac_val), 3), "\n")
```

```
TCPI: 1.429
```

```
R> cat("VAC: $", format(round(vac(bac, eac_typ))), big.mark = ","), "\n")
```

```
VAC: $ -225,000
```

The To-Complete Performance Index (TCPI) reports the cost efficiency required on all remaining work to finish within the original budget; values substantially above 1.0 signal an unrealistic recovery target.

4. Learning Curves

Sigmoidal (S-curve) models capture the characteristic pattern of project progress: slow start, rapid acceleration, and plateau approaching 100% completion. **PRA** provides three model types, Logistic, Pearl, and Gompertz, fitted via the Levenberg–Marquardt nonlinear least-squares algorithm in **minpack.lm** (Elzhov *et al.* 2016).

```
R> progress <- data.frame(
+   time      = 1:9,
+   completion = c(5, 15, 40, 60, 70, 75, 80, 85, 90)
+ )

R> fit_log <- fit_sigmoidal(progress, "time", "completion", "logistic")
R> fit_gomp <- fit_sigmoidal(progress, "time", "completion", "gompertz")
R>
R> cat("Logistic RSE: ", round(summary(fit_log)$sigma, 2), "\n")
```

Logistic RSE: 4.15

```
R> cat("Gompertz RSE: ", round(summary(fit_gomp)$sigma, 2), "\n")
```

Gompertz RSE: 2.89

```
R> plot_sigmoidal(fit_log, progress, "time", "completion", "logistic",
+                conf_level = 0.95,
+                main = "Logistic Learning Curve",
+                xlab = "Week", ylab = "Completion (%)")
```

`predict_sigmoidal()` generates forecasts with confidence bounds at arbitrary future time points:

```
R> preds <- predict_sigmoidal(fit_log, seq(10, 12, by = 0.5),
+                            "logistic", conf_level = 0.95)
R> print(round(preds, 1))
```

	x	pred	lwr	upr
1	10.0	84.3	78.2	90.3
2	10.5	84.3	78.2	90.4
3	11.0	84.3	78.2	90.4
4	11.5	84.3	78.2	90.5
5	12.0	84.3	78.2	90.5

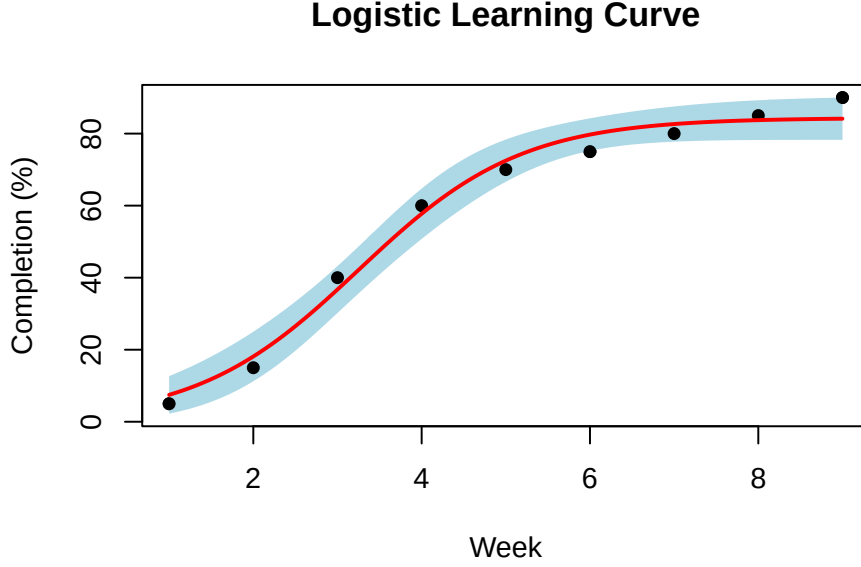


Figure 3: Logistic learning curve with 95% confidence band. The shaded region widens as extrapolation extends beyond the observed data range.

5. Bayesian Risk Inference

Project risks are frequently triggered by one or more underlying root causes whose presence is uncertain. **PRA** models this with a noisy-OR risk network (Pearl 2009): each root cause C_i acts on a risk event R independently, and R occurs if it is triggered by at least one cause. We illustrate with the schedule-delay risk in `building_project`, which has two candidate causes.

5.1. Prior Risk Probability

`risk_prob()` first computes each cause's marginal contribution by the law of total probability,

$$m_i = P(R \mid C_i) P(C_i) + P(R \mid \bar{C}_i) (1 - P(C_i)),$$

and then combines the independent causes with a noisy-OR,

$$P(R) = 1 - \prod_{i=1}^k (1 - m_i),$$

which always lies in $[0, 1]$ and reduces to the single-cause marginal when $k = 1$.

```
R> bp <- building_project
R> prior_delay <- risk_prob(bp$cause_probs, bp$risks_given_causes,
+                           bp$risks_given_not_causes)
R> cat("Prior P(schedule delay):", round(prior_delay, 3), "\n")
```

Prior P(schedule delay): 0.578

5.2. Posterior Risk Probability

When some causes are subsequently observed—for example, a site inspection confirms one cause is present while the other remains unknown—`risk_post_prob()` updates the estimate. An observed cause is fixed to its conditional probability, while an unobserved cause (coded NA) retains its marginal contribution m_i ; the causes are recombined with the same noisy-OR, so prior and posterior share a common scale:

$$P(R \mid \text{obs}) = 1 - \prod_{i=1}^k (1 - c_i), \quad c_i = \begin{cases} P(R \mid C_i) & C_i \text{ observed present,} \\ P(R \mid \bar{C}_i) & C_i \text{ observed absent,} \\ m_i & C_i \text{ unobserved.} \end{cases}$$

```
R> post_delay <- risk_post_prob(bp$cause_probs, bp$risks_given_causes,
+                               bp$risks_given_not_causes, bp$observed_causes)
R> cat("Posterior P(schedule delay):", round(post_delay, 3),
+      " (observed causes:", paste(bp$observed_causes, collapse = ", "), ")\n")
```

Posterior P(schedule delay): 0.797 (observed causes: 1, NA)

Observing an aggravating cause raises the posterior risk probability; observing a mitigating one lowers it. This closed-form update is inexpensive and complements the sample-based conditioning of the probabilistic network module (Section~7), which operates on full cost distributions rather than event probabilities.

6. Dependency Structure Analysis

Building on Govan and Damnjanovic (2016, 2020) and the foundational DSM literature (Steward 1981; Browning 2001), the DSM module quantifies structural coupling between project tasks through shared resources and risks.

6.1. Design Structure Matrices

The **Parent DSM** is derived from the Resource–Task matrix $\mathbf{S}$ (rows = resources, columns = tasks) as $\mathbf{P} = \mathbf{S}^\top \mathbf{S}$. Off-diagonal entry P_{jk} counts the number of resources shared between tasks j and k —the structural pathway through which disruptions propagate (Govan and Damnjanovic 2016).

```
R> S <- matrix(c(
+   1, 0, 1, 0,
+   1, 1, 0, 0,
+   0, 1, 0, 1,
+   0, 0, 1, 1,
+   0, 1, 1, 0
```

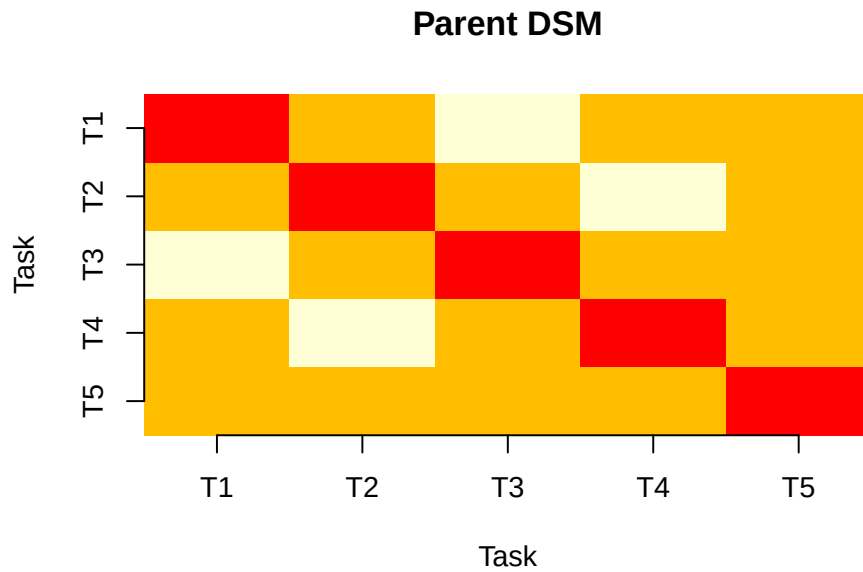


Figure 4: Parent DSM heatmap. Darker cells indicate more shared resources and stronger coupling between task pairs.

```
+ ), nrow = 4, ncol = 5)
R> rownames(S) <- paste0("R", 1:4)
R> colnames(S) <- paste0("T", 1:5)
```

```
R> p <- parent_dsm(S)
R> plot(p)
```

The **Grandparent DSM** adds a risk layer via the Risk–Resource matrix $\mathbf{R}$, tracing dependencies from risks through resources to tasks. Off-diagonal entry G_{jk} counts the number of risks shared between task j and task k through the resource chain—a direct operationalization of the structural network measures developed in [Govan and Damjanovic \(2020\)](#).

```
R> R_mat <- matrix(c(
+   1, 0, 1,
+   1, 1, 0,
+   0, 1, 0,
+   0, 0, 1
+ ), nrow = 3, ncol = 4)
R> rownames(R_mat) <- paste0("Risk", 1:3)
R> colnames(R_mat) <- paste0("R", 1:4)
```

```
R> g <- grandparent_dsm(S, R_mat)
R> plot(g)
```

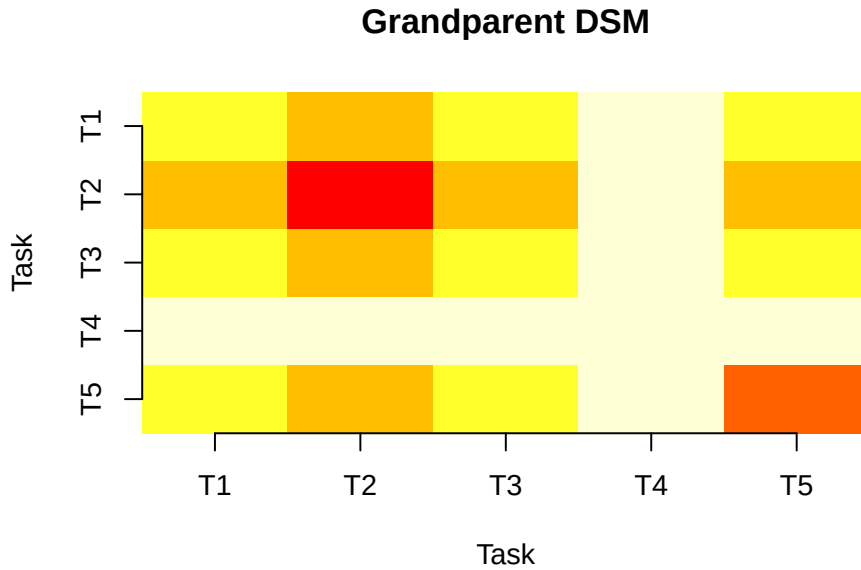


Figure 5: Grandparent DSM heatmap. Shows risk-based coupling: task pairs that share risk exposure through common resources tend to fail together.

Task pairs with high off-diagonal values in the Grandparent DSM represent concentrated risk exposure; these pairs are prime candidates for contingency buffering or resource decoupling.

7. Probabilistic Networks

This module is the secondary contribution of **PRA**: an open-source implementation—to the author’s knowledge the first—of the causal-network methods for project risk introduced by Govan (2014) and Govan and Damnjanovic (2016, 2020). The probabilistic network module extends DSM structural analysis by attaching probability distributions to each node, enabling forward simulation of cost uncertainty and Bayesian conditioning on observed evidence. A probabilistic network is a directed acyclic graph in which nodes represent project variables (risks, resources, tasks, and the project total) and edges encode conditional dependencies (Pearl 2009; Govan 2014). Where a DSM counts shared resources between task pairs, a probabilistic network propagates distributions through the causal chain from risks to total project cost.

7.1. Building a Network

`prob_net()` constructs the network from a node data frame, a links data frame, and a named list of distributions. Distributions may be `normal`, `lognormal`, `uniform`, `discrete`, or `conditional` (switching between two sub-distributions depending on a discrete parent node); `aggregate` nodes sum their parents.

```
R> nodes <- data.frame(
```

```

+   id = c("A", "B", "C", "D", "E", "F", "G", "H", "I"),
+   label = c("Risk-1", "Risk-2", "Resource-1", "Resource-2", "Resource-3",
+             "Task-1", "Task-2", "Task-3", "Project"),
+   stringsAsFactors = FALSE
+ )
R> links <- data.frame(
+   source = c("A", "B", "C", "D", "E", "F", "G", "H"),
+   target = c("C", "D", "F", "G", "H", "I", "I", "I"),
+   stringsAsFactors = FALSE
+ )
R> distributions <- list(
+   A = list(type = "discrete", values = c(1, 0), probs = c(0.70, 0.30)),
+   B = list(type = "discrete", values = c(1, 0), probs = c(0.60, 0.40)),
+   C = list(type = "conditional", condition = "A",
+     true_dist = list(type = "normal", mean = 30000, sd = 8000),
+     false_dist = list(type = "normal", mean = 15000, sd = 3000)),
+   D = list(type = "conditional", condition = "B",
+     true_dist = list(type = "normal", mean = 80000, sd = 20000),
+     false_dist = list(type = "normal", mean = 50000, sd = 10000)),
+   E = list(type = "normal", mean = 20000, sd = 4000),
+   F = list(type = "aggregate", nodes = "C"),
+   G = list(type = "aggregate", nodes = "D"),
+   H = list(type = "aggregate", nodes = "E"),
+   I = list(type = "aggregate", nodes = c("F", "G", "H"))
+ )
R> net <- prob_net(nodes, links, distributions = distributions)

```

7.2. Forward Simulation

`prob_net_sim()` draws samples from every node in topological order and returns a data frame with one column per node.

```

R> sim_net <- prob_net_sim(net, num_samples = 10000)
R> cat("Mean project cost: $", format(round(mean(sim_net$I)), big.mark = ","), "\n")

```

Mean project cost: \$ 113,612

```

R> cat("P80 project cost: $", format(round(quantile(sim_net$I, 0.80)),
+   big.mark = ","), "\n")

```

P80 project cost: \$ 135,543

7.3. Learning: Conditioning on Evidence

`prob_net_learn()` clamps observed nodes to fixed values and re-simulates the downstream distribution, an observational update in the sense of [Pearl \(2009\)](#). The following call fixes Risk-2 (node B) to absent (value 0), ruling out Technical Complexity as a concern:

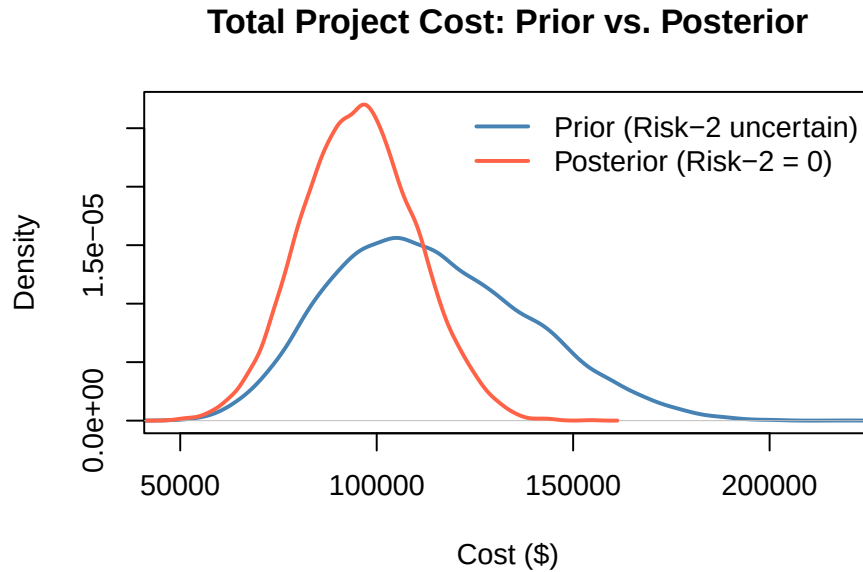


Figure 6: Total project cost before and after observing that Risk-2 (Technical Complexity) did not occur. Conditioning on Risk-2 = 0 eliminates the heavy upper tail and shifts the distribution leftward by roughly \$18,000 in expectation.

```
R> learn_net <- prob_net_learn(net, observations = list(B = 0),
+                               num_samples = 10000)
R>
R> prior_net_dens <- density(sim_net$I)
R> post_net_dens <- density(learn_net$I)
R>
R> plot(prior_net_dens, col = "steelblue", lwd = 2,
+       main = "Total Project Cost: Prior vs. Posterior",
+       xlab = "Cost ($)",
+       xlim = range(c(sim_net$I, learn_net$I)),
+       ylim = range(c(prior_net_dens$y, post_net_dens$y)))
R> lines(post_net_dens, col = "tomato", lwd = 2)
R> legend("topright",
+       legend = c("Prior (Risk-2 uncertain)", "Posterior (Risk-2 = 0)"),
+       col = c("steelblue", "tomato"), lwd = 2, bty = "n")
```

7.4. Intervention: The do-Operator

Conditioning answers “what if we *observe* that a risk did not occur.” A distinct question is “what if we *act* to eliminate it.” `prob_net_update()` performs structural surgery on the network—adding or removing edges and replacing node distributions—which corresponds to the *do*-operator in Pearl’s causal calculus (Pearl 2009), as opposed to `prob_net_learn()`, which conditions without altering the graph. Suppose management funds a mitigation that

decouples Resource-2 from Risk-2: remove the Risk-2 $\rightarrow$ Resource-2 edge and fix Resource-2 to its baseline cost.

```
R> inter_net <- prob_net_update(net,
+   remove_links = data.frame(source = "B", target = "D"),
+   update_distributions = list(
+     D = list(type = "normal", mean = 50000, sd = 10000)
+   )
+ )
R> do_net <- prob_net_sim(inter_net, num_samples = 10000)
R>
R> cat("Prior mean:           $", format(round(mean(sim_net$I)),   big.mark = ","), "\n")

Prior mean:           $ 113,612

R> cat("Seeing (Risk-2 = 0):   $", format(round(mean(learn_net$I)), big.mark = ","), "\n")

Seeing (Risk-2 = 0):   $ 95,547

R> cat("Doing  (do(Risk-2=0)): $", format(round(mean(do_net$I)),   big.mark = ","), "\n")

Doing  (do(Risk-2=0)): $ 95,616
```

Here *seeing* (`prob_net_learn()`) and *doing* (`prob_net_update()`) coincide because Risk-2 is an independent root cause with no shared upstream confounder, so conditioning on it and intervening on it induce the same downstream distribution. The two diverge, however, when a risk shares a common cause with project cost, as the next example shows.

7.5. When Seeing and Doing Diverge

Suppose a latent common cause—adverse site conditions—drives both a rework risk and, through a separate pathway, the foundation cost. Observing that rework did *not* occur is evidence that site conditions were favorable, which in turn lowers the expected foundation cost; *intervening* to prevent rework severs the risk but leaves site conditions, and hence the foundation cost, untouched. The network below encodes this structure: the common cause *U* feeds both the rework risk *Rk* and the foundation cost *Wf*, while the schedule cost *Vs* depends only on *Rk*.

```
R> set.seed(42)
R> cf_nodes <- data.frame(
+   id = c("U", "Rk", "Wf", "Vs", "Tc"), stringsAsFactors = FALSE)
R> cf_links <- data.frame(
+   source = c("U", "U", "Rk", "Wf", "Vs"),
+   target = c("Rk", "Wf", "Vs", "Tc", "Tc"), stringsAsFactors = FALSE)
R> cf_dists <- list(
+   U = list(type = "discrete", values = c(1, 0), probs = c(0.5, 0.5)),
```

```

+ Rk = list(type = "conditional", condition = "U",
+   true_dist = list(type = "discrete", values = c(1, 0), probs = c(0.9, 0.1)),
+   false_dist = list(type = "discrete", values = c(1, 0), probs = c(0.1, 0.9))),
+ Wf = list(type = "conditional", condition = "U",
+   true_dist = list(type = "normal", mean = 60000, sd = 5000),
+   false_dist = list(type = "normal", mean = 20000, sd = 5000)),
+ Vs = list(type = "conditional", condition = "Rk",
+   true_dist = list(type = "normal", mean = 40000, sd = 5000),
+   false_dist = list(type = "normal", mean = 10000, sd = 5000)),
+ Tc = list(type = "aggregate", nodes = c("Wf", "Vs")))
R> cf_net <- prob_net(cf_nodes, cf_links, distributions = cf_dists)
R>
R> cf_prior <- prob_net_sim(cf_net, num_samples = 10000)
R> cf_see <- prob_net_learn(cf_net, observations = list(Rk = 0),
+   num_samples = 10000)
R> cf_do <- prob_net_sim(prob_net_update(cf_net,
+   remove_links = data.frame(source = "U", target = "Rk"),
+   update_distributions = list(
+     Rk = list(type = "discrete", values = c(1, 0), probs = c(0, 1))),
+   num_samples = 10000)
R>
R> cat("Prior mean total:      $", format(round(mean(cf_prior$Tc)), big.mark = ","), "\n")

Prior mean total:      $ 65,003

R> cat("Seeing (Rk = 0) mean:    $", format(round(mean(cf_see$Tc)), big.mark = ","),
+   "    E[U | Rk=0] =", round(mean(cf_see$U), 2), "\n")

Seeing (Rk = 0) mean:    $ 33,936    E[U | Rk=0] = 0.1

R> cat("Doing do(Rk = 0) mean:  $", format(round(mean(cf_do$Tc)), big.mark = ","),
+   "    E[U] =", round(mean(cf_do$U), 2), "\n")

Doing do(Rk = 0) mean:  $ 49,798    E[U] = 0.49

```

Conditioning propagates the evidence back through the shared cause—the expected site-condition indicator falls from its prior of 0.5 to about 0.1—so *seeing* that rework did not occur pulls the foundation cost down as well and yields a markedly lower expected total than *doing*. Because `prob_net_learn()` conditions by rejection sampling over whole joint draws, it captures this backdoor propagation that a clamp-and-forward update cannot; this is precisely where observation and intervention must be distinguished, and where the *do*-operator becomes indispensable.

7.6. Ranking Risks by Intervention

Repeating the intervention for each root risk and measuring the reduction in project-cost variance yields a principled, network-level mitigation ranking—a causal analog of the Tornado chart of Section~2:

Risk Importance: Variance Eliminated by do(risk = 0)

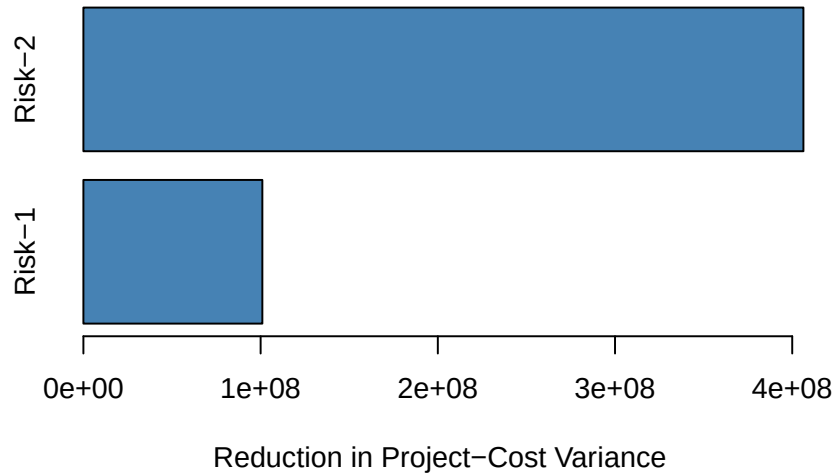


Figure 7: Risk importance for the probabilistic network: the reduction in total project-cost variance achieved by intervening to eliminate each root risk. Taller bars indicate higher-priority mitigation targets.

```
R> base_var <- var(sim_net$I)
R>
R> do_A <- prob_net_sim(prob_net_update(net,
+   remove_links = data.frame(source = "A", target = "C"),
+   update_distributions = list(C = list(type = "normal", mean = 15000, sd = 3000))),
+   num_samples = 10000)
R>
R> importance <- c(
+   "Risk-1" = base_var - var(do_A$I),
+   "Risk-2" = base_var - var(do_net$I)
+ )
R> barplot(sort(importance), horiz = TRUE, col = "steelblue",
+   main = "Risk Importance: Variance Eliminated by do(risk = 0)",
+   xlab = "Reduction in Project-Cost Variance")
```

The probabilistic network module is newer than the analytical core, and its API may still evolve in future releases.

8. Model Context Protocol Integration

Version 0.5.0 of **PRA** exposes all analytical functions as tools callable by large-language-model (LLM) agents through the Model Context Protocol (MCP) ([Anthropic 2024](#)). `pra_mcp_server()` starts a server that advertises each tool—Monte Carlo simulation, EVM, Bayesian risk, learn-

ing curves, DSM, and the rest, so that Claude, Claude Code, and other MCP-compatible clients can invoke them as native tools. The server is registered in Claude Code with a single shell command:

```
R> claude mcp add -s project pra -- Rscript -e "PRA::pra_mcp_server()"
```

For Claude Desktop, add the following block to the MCP configuration file:

```
{
  "mcpServers": {
    "pra": { "command": "Rscript",
              "args": ["-e", "PRA::pra_mcp_server()"] }
  }
}
```

The tool definitions are built once with `ellmer::tool()` (Wickham *et al.* 2025) and served by **mcptools**. Each tool validates its JSON inputs and returns a plain-text summary for the LLM together with, where applicable, a rich HTML result with inline plots for clients that render it.

To help an agent decide *when* and *how* to call each tool, **PRA** bundles a set of Agent Skills under `system.file("skills", package = "PRA")`. Each skill is a `SKILL.md` file with structured guidance, when to use the method, which tool to call, the JSON argument format, and how to interpret the result, covering Monte Carlo simulation, sensitivity and contingency, EVM, Bayesian risk, learning curves, and dependency structure analysis, plus a routing overview. Pairing the skills with the MCP server lets an agent carry out a complete project risk workflow (for example, `mcs_tool` followed by `contingency_tool` and `sensitivity_tool`) from a natural-language request, without embedding an LLM in the package itself.

The MCP layer is strictly optional: every analytical function operates without any optional dependency. Only `pra_mcp_server()` requires **ellmer**, **mcptools**, and **jsonlite**.

9. Illustrative Case Study

This section demonstrates an integrated, end-to-end project risk analysis for the `building_project` dataset introduced in Section~2, progressing through uncertainty quantification, risk-driver ranking, performance monitoring, completion forecasting, Bayesian risk updating, dependency structure analysis, and network cost propagation in a single R session. The workflow reuses objects built in the preceding sections (for example the closed-form task moments, the fitted learning curve, and the probabilistic network), illustrating how the modules compose within one analysis.

9.1. Step 1: Characterize Task Uncertainty

Combine the rapid analytical estimate from the Second Moment Method—reusing the closed-form triangular moments `task_means` and `task_vars` from Section~2 with the dataset correlation matrix—with the full simulated distribution from Monte Carlo simulation:

```

R> bp <- building_project
R> set.seed(42)
R>
R> smm_cs <- smm(task_means, task_vars, bp$cor_mat) # analytic, correlated
R> sim      <- mcs(10000, bp$task_distributions)    # full distribution
R> res      <- contingency(sim, phigh = 0.80, pbase = 0.50)
R>
R> cat("SMM total (correlated): mean", round(smm_cs$total_mean, 1),
+      "SD", round(smm_cs$total_std, 2), "weeks\n")

SMM total (correlated): mean 71.7 SD 6.59 weeks

R> cat("P50 schedule:", round(quantile(sim$total_distribution, 0.50), 1), "weeks\n")

P50 schedule: 71.6 weeks

R> cat("P80 schedule:", round(quantile(sim$total_distribution, 0.80), 1), "weeks\n")

P80 schedule: 75.6 weeks

R> cat("Contingency (P80-P50):", round(res, 2), "weeks\n")

Contingency (P80-P50): 3.98 weeks

```

9.2. Step 2: Rank Risk Drivers

`sensitivity()` ranks the tasks by their share of total schedule variance, focusing mitigation on the dominant drivers:

```

R> drivers <- sensitivity(bp$task_distributions)
R> names(drivers) <- bp$task_names
R> print(round(sort(drivers, decreasing = TRUE), 3))

```

Structural Frame Mechanical/Electrical/Plumbing	
0.304	0.206
Interior Fit-out & Finishes	Building Envelope
0.206	0.133
Site Preparation & Earthworks	Foundations
0.076	0.076

9.3. Step 3: Monitor Earned Value at Mid-Project

```
R> pv_cs <- pv(bp$bac, bp$schedule, bp$time_period)
R> ev_cs <- ev(bp$bac, bp$actual_per_complete)
R> ac_cs <- ac(bp$actual_costs, bp$time_period, cumulative = FALSE)
R>
R> cat("SPI:", round(spi(ev_cs, pv_cs), 3),
+      " CPI:", round(cpi(ev_cs, ac_cs), 3), "\n")

SPI: 0.889  CPI: 0.986

R> cat("EAC (typical): $", format(round(eac(bp$bac,
+      method = "typical", cpi = cpi(ev_cs, ac_cs))), big.mark = ","), "\n")

EAC (typical): $ 8,625,000
```

9.4. Step 4: Forecast Completion

Reusing the logistic learning curve `fit_log` fitted in Section~4, project percent-complete at future weeks with confidence bounds:

```
R> preds_cs <- predict_sigmoidal(fit_log, seq(10, 12, by = 1),
+                               "logistic", conf_level = 0.95)
R> print(round(preds_cs, 1))

  x pred  lwr  upr
1 10 84.3 78.2 90.3
2 11 84.3 78.2 90.4
3 12 84.3 78.2 90.5
```

9.5. Step 5: Update Risk Probability After Site Inspection

```
R> prior_r <- risk_prob(bp$cause_probs, bp$risks_given_causes,
+                      bp$risks_given_not_causes)
R> post_r <- risk_post_prob(bp$cause_probs, bp$risks_given_causes,
+                          bp$risks_given_not_causes, bp$observed_causes)
R> cat("Prior P(delay):", round(prior_r, 3),
+      "-> Posterior P(delay):", round(post_r, 3), "\n")

Prior P(delay): 0.578 -> Posterior P(delay): 0.797
```

9.6. Step 6: Identify Coupled Tasks via DSM

```
R> g_cs <- grandparent_dsm(bp$resource_task, bp$risk_resource)
R> plot(g_cs)
```

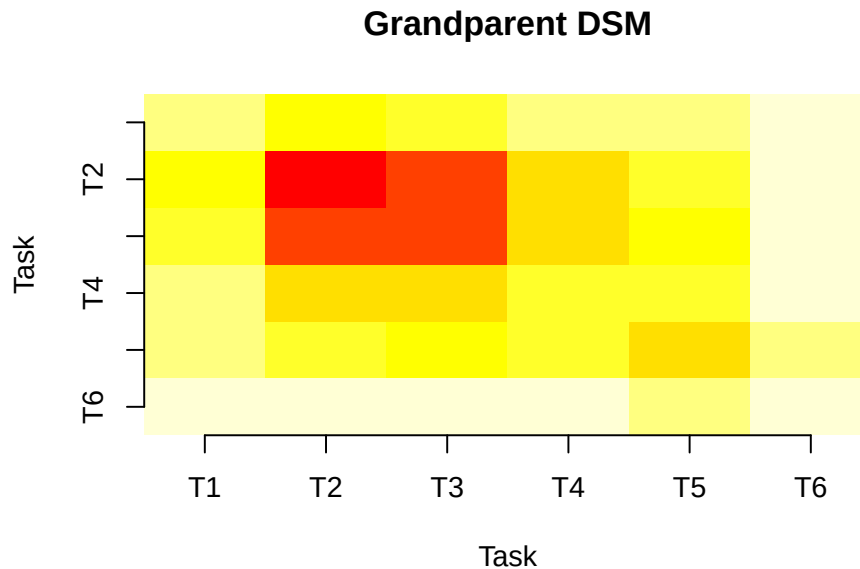


Figure 8: Grandparent DSM for the case-study project, showing which task pairs share the most risk exposure through common resources.

The Grandparent DSM reveals which task pairs share risk exposure through common resources, informing targeted contingency allocation and resource decoupling decisions.

9.7. Step 7: Propagate Cost Through the Network

Finally, use the probabilistic network `net` from Section~7 to propagate cost uncertainty to the project total, then condition on the observation that Risk-2 (Technical Complexity) did not occur:

```
R> cat("Prior mean project cost: $",
+      format(round(mean(sim_net$I)), big.mark = ","), "\n")
```

```
Prior mean project cost: $ 113,612
```

```
R> cat("Prior P80 project cost: $",
+      format(round(quantile(sim_net$I, 0.80)), big.mark = ","), "\n")
```

```
Prior P80 project cost: $ 135,543
```

```
R> cat("Posterior mean (Risk-2 = 0): $",
+      format(round(mean(learn_net$I)), big.mark = ","), "\n")
```

```
Posterior mean (Risk-2 = 0): $ 95,547
```

Conditioning on the resolved risk lowers the expected project cost, closing the loop from uncertainty characterization to evidence-based updating.

10. Summary

PRA is a CRAN-available R package whose primary contribution is being the first of its kind to unify the foundational methods of quantitative project risk analysis in a single, open-source framework of ten integrated analytical modules: SMM, MCS, contingency analysis, sensitivity analysis, EVM (11 metrics), Bayesian risk inference and updating, cost risk distributions, sigmoidal learning curves, correlation matrix construction, and design structure matrices. As a secondary contribution, an eleventh module delivers an open-source implementation—to the author’s knowledge the first—of the causal-network and probabilistic-network methods for project risk developed in Govan (2014) and Govan and Damnjanovic (2016, 2020), with forward simulation and Bayesian conditioning.

As a third contribution, version 0.5.0 exposes every analytical function as a tool over the Model Context Protocol, complemented by a bundled set of Agent Skills, so that LLM agents can drive the package without any embedded model—the first project risk analysis package to do so. All exported functions adhere to rOpenSci SRR standards, with a test suite verifying correctness, edge-case handling, and numerical stability across three operating systems. The package is licensed under MIT and the source is available at <https://github.com/paulgovan/PRA>; documentation and a companion online book are at <https://paulgovan.github.io/PRA/>.

10.1. Limitations and Future Work

Several limitations apply. First, **PRA** currently propagates uncertainty at the task level and does not yet model schedule logic—critical path, activity precedence, and resource leveling. Full schedule-network risk analysis, in which schedule Monte Carlo runs over an activity-precedence network, such as Program Evaluation and Review Technique / Critical Path Method (PERT/CPM) to identify the critical path and criticality indices under uncertainty, is a planned future direction that would extend **PRA** from task-level to network-level schedule risk. Second, `mcs()` induces task correlation by applying a Cholesky factor to the sampled values, which reproduces the target rank correlation but distorts the marginal means; the examples in this article therefore treat tasks as independent for Monte Carlo simulation and use `smm()`, which handles correlation analytically, when correlation matters. A copula-based implementation that preserves the marginals is planned. Third, the probabilistic network module is newer than the analytical core: its API may still evolve, and it does not yet provide convergence diagnostics for the Monte Carlo estimators. Fourth, the package currently accepts data as R vectors and lists; direct import from project management file formats (Microsoft Project, Primavera P6, CSV exports from Jira or Asana) is not supported and is planned for a future release.

Computational details

The results in this article were produced using R (R Core Team 2024) with the **PRA** package (version 0.5.0) and its hard dependencies **mc2d** (Pouillot and Delignette-Muller 2010) and

minpack.lm (Elzhov *et al.* 2016). The optional Model Context Protocol server additionally uses **ellmer** (Wickham *et al.* 2025), **mcptools**, and **jsonlite**. R itself and all packages used are available from the CRAN at <https://CRAN.R-project.org/>. A random seed of 42 is set at the start of the session so that the simulation output is reproducible.

References

- Anthropic (2024). “Model Context Protocol.” URL <https://modelcontextprotocol.io>.
- Benjamin JR, Cornell CA (2000). *Probability, Statistics, and Decision for Civil Engineers*. Dover Publications, Mineola, NY.
- Browning TR (2001). “Applying the Design Structure Matrix to System Decomposition and Integration Problems: A Review and New Directions.” *IEEE Transactions on Engineering Management*, **48**(3), 292–306. doi:10.1109/17.946528.
- Bürkner PC (2017). “**brms**: An R Package for Bayesian Multilevel Models Using Stan.” *Journal of Statistical Software*, **80**(1), 1–28. doi:10.18637/jss.v080.i01.
- Damnjanovic I, Reinschmidt K (2020). *Data Analytics for Engineering and Construction Project Risk Management*. Springer, Cham, Switzerland. doi:10.1007/978-3-030-14251-3.
- Elzhov TV, Mullen KM, Spiess AN, Bolker B (2016). *minpack.lm: R Interface to the Levenberg-Marquardt Nonlinear Least-Squares Algorithm Found in MINPACK*. URL <https://CRAN.R-project.org/package=minpack.lm>.
- Fleming QW, Koppelman JM (2010). *Earned Value Project Management*. 4 edition. Project Management Institute, Newtown Square, PA.
- Gonçalves-Dosantos JC, García-Jurado I, Costa J (2022). *ProjectManagement: Management of Deterministic and Stochastic Projects*. R package, URL <https://CRAN.R-project.org/package=ProjectManagement>.
- Govan P (2014). *A Resource-Based View on Project Risk Management*. Ph.D. thesis, The University of Texas at Austin.
- Govan P, Damnjanovic I (2016). “The resource-based view on project risk management.” *Journal of Construction Engineering and Management*, **142**(9), 04016034. doi:10.1061/(ASCE)CE.1943-7862.0001157.
- Govan P, Damnjanovic I (2020). “Structural Network Measures for Risk Assessment of Construction Projects.” *ASCE-ASME Journal of Risk and Uncertainty in Engineering Systems, Part A: Civil Engineering*. doi:10.1061/AJRUA6.0001041.
- Iooss B, Da Veiga S, Janon A, Pujol G (2024). *sensitivity: Global Sensitivity Analysis of Model Outputs*. URL <https://CRAN.R-project.org/package=sensitivity>.
- Kelley D (2023). *plan: Tools for Project Planning*. R package, URL <https://CRAN.R-project.org/package=plan>.

- Luedeling E, Goehring L, Schiffers K, Whitney C, Fernandez E (2023). *decisionSupport: Quantitative Support of Decision Making under Uncertainty*. R package, URL <https://CRAN.R-project.org/package=decisionSupport>.
- Pearl J (2009). *Causality: Models, Reasoning, and Inference*. 2nd edition. Cambridge University Press, Cambridge, UK.
- Pouillot R, Delignette-Muller ML (2010). “Evaluating Variability and Uncertainty in Microbial Quantitative Risk Assessment Using Two R Packages.” *International Journal of Food Microbiology*, **142**(3), 330–340. doi:[10.1016/j.ijfoodmicro.2010.07.011](https://doi.org/10.1016/j.ijfoodmicro.2010.07.011).
- Project Management Institute (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. 7th edition. Project Management Institute, Newtown Square, PA.
- R Core Team (2024). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL <https://www.R-project.org/>.
- rOpenSci Statistical Software Peer Review (2021). *Statistical Software Peer Review: Standards and Guidelines*. URL <https://stats-devguide.ropensci.org>.
- Stan Development Team (2024). “**RStan**: The R Interface to Stan.” URL <https://mc-stan.org/>.
- Steward DV (1981). *Systems Analysis and Management: Structure, Strategy and Design*. Petrocelli Books, New York, NY.
- Vose D (2008). *Risk Analysis: A Quantitative Guide*. 3 edition. Wiley, Chichester.
- Wickham H, et al. (2025). *ellmer: Chat with Large Language Models*. URL <https://ellmer.tidyverse.org>.

Affiliation:

Journal of Statistical Software

published by the Foundation for Open Access Statistics

MMMMMM YYYY, Volume VV, Issue II

doi:[10.18637/jss.v000.i00](https://doi.org/10.18637/jss.v000.i00)

<http://www.jstatsoft.org/>

<http://www.foastat.org/>

Submitted: yyyy-mm-dd

Accepted: yyyy-mm-dd